CS641 – Introduction to Computer Vision

Lecture 03: Image Filtering

Dr. Abdul Basit — Associate Professor, CSIT, University of Balochistan

May 4, 2026

Topics Covered

Section	Topics
1. Image Noise	What is noise? Mathematical model
2. Removing Image Noise	Averaging, Weighted Average, Moving Average 2D
3. Correlation	Box Filter, Gaussian Filter
Python Code	Hands-on implementation with OpenCV + NumPy

1. Image Noise

Whenever we capture a photograph or scan a document, the resulting image is rarely a perfect copy of reality. Small random errors creep in — these errors are collectively called **image noise**. Think of it like this: imagine you are speaking a message through a noisy crowd. The person on the other end hears your voice **plus** all the background noise. Similarly, a camera captures the true scene **plus** random interference from electronics, lighting, sensor imperfections, etc.

1.1 Mathematical Definition

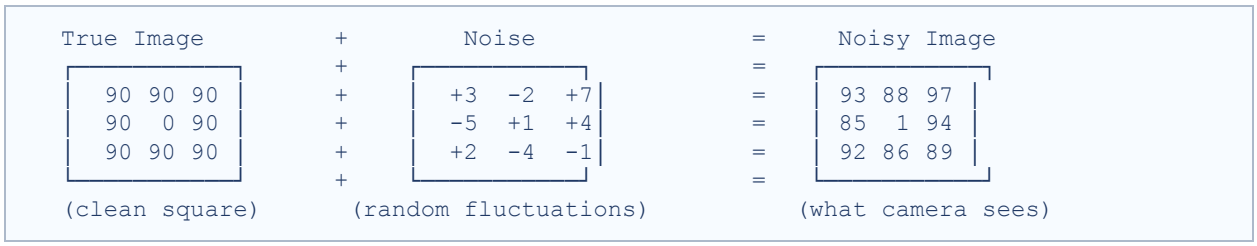
Mathematically, every pixel in a noisy image **Î(x, y)** is the sum of the true pixel value **I(x, y)** and a noise term **n(x, y)**:

$$\hat{I}(x, y) = I(x, y) + n(x, y)$$

Where:

- **Î(x, y)** — the noisy (observed) image pixel at position (x, y)
- **I(x, y)** — the true (clean) pixel value we wish we had
- **n(x, y)** — the noise: a random number added to each pixel independently

1.2 Visual Analogy



1.3 Key Properties of Noise

- **Independence:** The noise added to pixel A is completely unrelated to the noise added to pixel B. Each pixel's error is rolled separately, like individual dice throws.
- **True values are locally similar:** The actual brightness of nearby pixels in the real world is usually close — a clear sky is uniformly blue, a white wall is uniformly white, etc. Noise is the only thing making them look different from each other.
- **Types of noise:** Gaussian noise (most common — bell-curve distribution), Salt-and-pepper noise (random black/white pixels), Poisson noise (from photon counting in cameras).

Why This Matters

Understanding that noise is (1) random, (2) independent per pixel, and (3) added on top of true values that are locally smooth — is the foundation of EVERY noise removal technique we will study.

2. Removing Image Noise

Since we know the noise is **random and independent**, and the true values are **locally similar**, the most natural idea is: **average a pixel with its neighbours**. The true signal reinforces; the random noise cancels out.

2.1 Averaging (1D Case First)

Let's understand the idea in 1D (a single row of pixels) before jumping to 2D images.

```
Original noisy signal (pixel row):
──────────────────────────────────────────────────────────────
Pos: 0  1  2  3  4  5  6  7  8  9  10 11 12
Val: 85 92 78 95 88 102 79 91 84 97  88 105 82
                          (irregular, jumpy = noisy)

After averaging each pixel with 2 neighbours on each side:
──────────────────────────────────────────────────────────────
Each output = (prev2 + prev1 + self + next1 + next2) / 5
Result:  88  90  90  91  91   91   89  91  90  91  90  91  87
                          (smoother = noise reduced)
```

> **Intuition**
> If the true value at position 5 is 90, noise makes it 102. But the neighbours (79, 91, 97...) also have small random errors. When we AVERAGE them together, the positive errors and negative errors partially cancel out, giving us something closer to 90.

2.2 Why Does Averaging Work? (The Math Intuition)

There are two key assumptions that make averaging valid:

- **Assumption 1 — Local smoothness:** The **true** value of a pixel is probably very close to the true values of its neighbours. A sky pixel and its 8 neighbours all represent 'blue sky', so their true values are nearly identical.
- **Assumption 2 — Independent noise:** The noise on each pixel is random and independent. If noise has zero mean (equally likely to be positive or negative), averaging N noisy measurements of the same quantity reduces noise by a factor of $\sqrt{N}$.

$$\text{noise\_after\_averaging} \approx \text{noise\_per\_pixel} / \sqrt{(\text{window\_size})}$$

So a 3×3 window (9 pixels) reduces noise by $\sqrt{9} = 3\times$. A 5×5 window reduces it by $\sqrt{25} = 5\times$. **Larger window → more noise reduction, but also more blurring.**

2.3 Weighted Average

The simple average treats all neighbours equally. But intuitively, **the closer a pixel is, the more similar it should be to the centre pixel**. So we can give **higher weight to closer neighbours** and lower weight to distant ones.

Uniform weights (Box / Simple average)

All 5 neighbours (including self) get equal weight 1/5:

Weights: [1 1 1 1 1] / 5

```
Signal:     ... 88   95   102   84   97 ...
                 ↑    ↑     ↑    ↑    ↑
Weights:     1/5  1/5   1/5  1/5  1/5
                           ↓
Output at centre: (88+95+102+84+97)/5 = 93.2
```

Non-Uniform / Gaussian-like weights

Closer pixels get higher weight. For a 5-element window [1, 4, 6, 4, 1]/16:

Weights: [1 4 6 4 1] / 16

```
Signal:     ... 88   95   102   84   97 ...
                 ↑    ↑     ↑    ↑    ↑
Weights:     1/16  4/16  6/16  4/16  1/16
                           ↓
Output = (1×88 + 4×95 + 6×102 + 4×84 + 1×97) / 16
       = (88 + 380 + 612 + 336 + 97) / 16
       = 1513 / 16  ≈  94.6  (centre dominates!)
```

Weighting	Effect
Uniform [1,1,1,1,1]/5	Simple average. All neighbours equal. Fast but can oversmooth.
Non-uniform [1,4,6,4,1]/16	Centre has 6× more weight than edges. Better edge preservation.
Gaussian curve	Continuous version: weight falls off smoothly with distance.

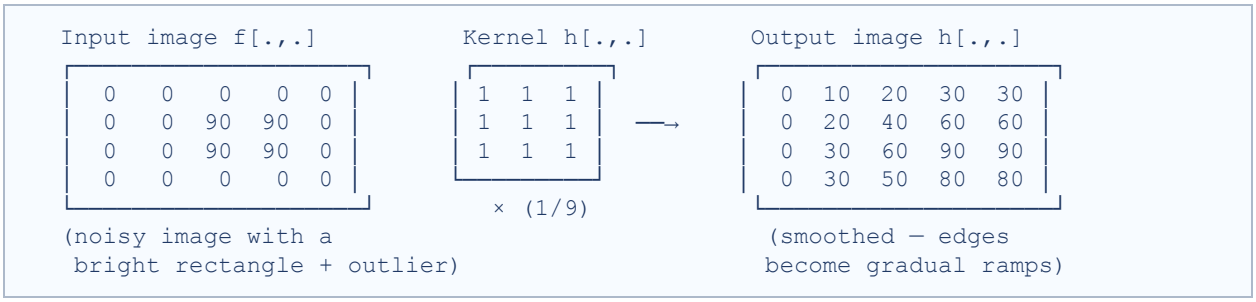
2.4 Moving Average in 2D (Images)

A real image is 2D — pixels have both row (x) and column (y) coordinates. We extend the 1D moving average to 2D by averaging a pixel with its **rectangular neighbourhood** (a square patch around it). This patch is called the **filter window** or **kernel**.

How the sliding window works

Think of a small square (the kernel) sliding across the image. At each position:

- Place the kernel centre over the target pixel.
- Multiply each kernel weight by the corresponding image pixel.
- Sum all results and write the sum into the output image at that position.
- Slide the kernel one step and repeat for every pixel.



Key Observation from the Diagram

Notice how the sharp 0-to-90 edge in the input becomes a gradual 0→10→20→30→90 ramp in the output. Noise reduction always comes with some blurring of sharp edges. This is the fundamental trade-off in filtering.

Step-by-step manual calculation

Let's trace through one output pixel calculation. Consider the 3×3 box filter (weights = 1/9 each):

Target pixel at row 3, col 3 (the '?' position): Its 3×3 neighbourhood is:

Neighbourhood patch:

90	0	90
90	**90**	90
0	0	0

```
output = (90+0+90 + 90+90+90 + 0+0+0) / 9 = 450/9 = 50
```

So the output at that position is 50 — a blend of the bright rectangle and the dark background.

3. Correlation (The Formal Framework)

So far we talked about 'averaging with weights'. Correlation is the **formal mathematical name** for this operation. It generalises averaging to allow **any set of weights** (not just uniform 1/N).

3.1 Correlation with Uniform Weights

The uniform-weight correlation formula for output pixel h(i, j):

$$h(i,j) = 1/(2n+1)^2 \times \Sigma[k=-n \text{ to } n]\ \Sigma[l=-n \text{ to } n]\ f(i+k, j+l)$$

Where **n** determines window size: a window of (2n+1) × (2n+1). For example:

n value	Window size
n = 0	1×1 (no change, just copying)
n = 1	3×3 (9 pixels, most common)
n = 2	5×5 (25 pixels, heavier smoothing)
n = 3	7×7 (49 pixels)

> **Window Size is Always ODD**
> The kernel must have a well-defined centre pixel. A 3×3 kernel has centre at (1,1). A 4×4 has no exact centre — so kernel sizes are always odd: 3×3, 5×5, 7×7, ...

3.2 Correlation with Non-Uniform Weights (General)

Now generalise to allow **different weights** for different positions in the window. The kernel/mask **g(k, l)** replaces the uniform $1/(2n+1)^2$:

$$h(i,j) = \Sigma[k,l]\ g(k,l) \times f(i+k, j+l)$$

This is also called **cross-correlation**, written as $h = g \otimes f$ (or $g \star f$). **g(k, l)** is the **kernel or mask** — the matrix of weights. **f(i+k, j+l)** are the pixel values from the input image at offset (k, l) from position (i, j).

Term	Meaning
g(k, l)	Kernel / Mask / Filter — the matrix of weights (e.g. 3×3 Gaussian)
f(i+k, j+l)	Input image pixel at offset (k,l) from the output position
h(i, j)	Output (filtered) image pixel at position (i, j)
h = g ⊗f	Shorthand: output = kernel cross-correlated with input

3.3 The Box Filter (Averaging Filter)

The simplest kernel is the **box filter**: a square matrix where every entry is 1/(size²). For a 3×3 box filter:

3 × 3 Box Filter Kernel g[·,·]:

1	1	1
1	1	1
1	1	1

Box filter × 1/9

Effect: replaces each pixel with the plain average of itself and its 8 neighbours.

- **Pro:** Very fast (uniform weights, easily optimised)
- **Con:** Creates a 'blocky' blurring effect — visible ringing artefacts at edges

Visual Effect

Apply a box filter to a sharp edge (black on left, white on right) and you get a gradient staircase. Apply Gaussian and you get a smooth gradual transition. Gaussian looks more 'natural'.

3.4 The Gaussian Filter

The Gaussian filter uses weights based on the **Gaussian (normal) distribution** — a bell-curve shape. Pixels closer to the centre get **exponentially higher** weight than distant ones.

The Gaussian Function Formula

$$G_\sigma(x,y) = (1 / 2\pi\sigma^2) \times \exp(-(x^2+y^2) / 2\sigma^2)$$

Where **σ (sigma)** is the **standard deviation** — it controls how wide/narrow the bell curve is:

- **Small σ** → narrow bell → only very close neighbours matter → gentle blur
- **Large σ** → wide bell → distant neighbours still contribute → heavy blur

Example: 3×3 Gaussian Kernel (approximate, σ=1)

1	2	1
2	4	2
1	2	1

3×3 Gaussian (σ≈1) × 1/16

The centre weight (4) is highest. Edge-centre weights (2) are less. Corner weights (1) are least. Total = 16, so divide by 16.

Example: 5×5 Gaussian Kernel (σ=1)

The actual float kernel values for a 5×5 Gaussian with σ=1:

```
0.003  0.013  0.022  0.013  0.003
0.013  0.059  0.097  0.059  0.013
0.022  0.097  0.159  0.097  0.022   ← centre has highest weight
0.013  0.059  0.097  0.059  0.013
0.003  0.013  0.022  0.013  0.003
All values sum to 1.0 (normalised)
```

Box Filter vs Gaussian Filter — Visual Comparison

Box Filter	Gaussian Filter
Uniform weights: 1/9 each	Non-uniform: centre has highest weight
Sharp blocky blur	Smooth natural-looking blur
Creates ringing artefacts near edges	No ringing — smooth transition
Faster (all weights equal)	Slightly more computation
Not rotationally symmetric	Rotationally symmetric (same in all directions)
Kernel: all 1s ÷ N	Kernel: Gaussian bell curve values

4. Python Code — Hands-On Implementation

Below are complete, well-commented Python scripts you can run in your **cv_env** conda environment using OpenCV and NumPy.

4.1 Setup — Activating the Environment

```
# In terminal (Fedora / any Linux):
conda activate cv_env
python your_script.py

# Required libraries:
import cv2          # OpenCV — image I/O and filters
import numpy as np  # NumPy — array operations
import matplotlib.pyplot as plt  # Plotting results
```

4.2 Image Noise — Adding Noise Manually

```
import cv2
import numpy as np
import matplotlib.pyplot as plt

# ──────────────────────────────────────────────────────
# STEP 1: Load or create a clean image
# ──────────────────────────────────────────────────────

# Option A: Load from disk
img = cv2.imread('image.jpg', cv2.IMREAD_GRAYSCALE)  # grayscale

# Option B: Create a synthetic image (rectangle on black bg)
img = np.zeros((200, 200), dtype=np.uint8)   # all black, 200x200
img[50:150, 50:150] = 90                      # white rectangle

# ──────────────────────────────────────────────────────
# STEP 2: Create Gaussian noise
# Formula: noisy = true_image + noise
# n(x,y) ~ Normal(mean=0, std=sigma)
```

```
# ──────────────────────────────────────────────────────────
sigma = 20  # noise strength (higher = more noise)
noise = np.random.normal(loc=0, scale=sigma, size=img.shape)

# ──────────────────────────────────────────────────────────
# STEP 3: Add noise to image
# Must convert to float first to avoid overflow
# Then clip back to valid range [0, 255] and convert to uint8
# ──────────────────────────────────────────────────────────
img_float = img.astype(np.float64)    # convert to float
noisy_float = img_float + noise        # add noise
noisy = np.clip(noisy_float, 0, 255).astype(np.uint8)  # clip & convert

# ──────────────────────────────────────────────────────────
# STEP 4: Display both images side by side
# ──────────────────────────────────────────────────────────
fig, axes = plt.subplots(1, 2, figsize=(10, 4))
axes[0].imshow(img, cmap='gray', vmin=0, vmax=255)
axes[0].set_title('Clean Image (True Signal)')
axes[0].axis('off')

axes[1].imshow(noisy, cmap='gray', vmin=0, vmax=255)
axes[1].set_title(f'Noisy Image (sigma={sigma})')
axes[1].axis('off')

plt.tight_layout()
plt.savefig('noise_demo.png', dpi=150)
plt.show()
print('Saved: noise_demo.png')
```

4.3 Box Filter (Averaging Filter) with OpenCV

```
import cv2
```

```
import numpy as np
import matplotlib.pyplot as plt

# ——————————————————————————————————————————
# CREATING A BOX FILTER KERNEL MANUALLY
# ——————————————————————————————————————————

# 3x3 box filter kernel: every weight = 1/9
box_kernel_3x3 = np.ones((3, 3), dtype=np.float32) / 9.0
print('3x3 Box Kernel:')
print(box_kernel_3x3)
# Output:
# [[0.111 0.111 0.111]
#  [0.111 0.111 0.111]
#  [0.111 0.111 0.111]]

# 5x5 box filter kernel: every weight = 1/25
box_kernel_5x5 = np.ones((5, 5), dtype=np.float32) / 25.0

# ——————————————————————————————————————————
# APPLYING BOX FILTER USING cv2.filter2D (manual kernel)
# ——————————————————————————————————————————

img = cv2.imread('image.jpg', cv2.IMREAD_GRAYSCALE)
# Add noise first
noise = np.random.normal(0, 20, img.shape).astype(np.int16)
noisy = np.clip(img.astype(np.int16) + noise, 0, 255).astype(np.uint8)

# Apply box filter manually (using our kernel + cv2.filter2D)
# cv2.filter2D performs 2D correlation (not convolution)
# ddepth=-1 means output has same depth as input
filtered_manual = cv2.filter2D(noisy, ddepth=-1, kernel=box_kernel_3x3)

# ——————————————————————————————————————————
# APPLYING BOX FILTER USING cv2.blur (built-in shortcut)
# cv2.blur(image, (kernel_width, kernel_height))
# ——————————————————————————————————————————
```

```
filtered_3x3 = cv2.blur(noisy, (3, 3))
filtered_5x5 = cv2.blur(noisy, (5, 5))
filtered_9x9 = cv2.blur(noisy, (9, 9))

# ──────────────────────────────────────────────────────────
# VISUALISE: Effect of different box filter sizes
# ──────────────────────────────────────────────────────────

fig, axes = plt.subplots(1, 5, figsize=(18, 4))
images = [img, noisy, filtered_3x3, filtered_5x5, filtered_9x9]
titles = ['Original', 'Noisy (σ=20)',
          'Box 3×3', 'Box 5×5', 'Box 9×9']

for ax, image, title in zip(axes, images, titles):
    ax.imshow(image, cmap='gray', vmin=0, vmax=255)
    ax.set_title(title, fontsize=11)
    ax.axis('off')

plt.tight_layout()
plt.savefig('box_filter_comparison.png', dpi=150)
plt.show()
```

4.4 Weighted Average (Manual 1D)

```
import numpy as np

# ──────────────────────────────────────────────────────────
# 1D WEIGHTED AVERAGE — showing uniform vs non-uniform
# ──────────────────────────────────────────────────────────

# Noisy 1D signal
signal = np.array([85, 92, 78, 95, 88, 102, 79, 91, 84, 97, 88, 105, 82],
                  dtype=np.float64)

# Uniform weights [1,1,1,1,1]/5
```

```
w_uniform = np.array([1, 1, 1, 1, 1], dtype=np.float64) / 5.0

# Non-uniform weights [1,4,6,4,1]/16
w_nonuniform = np.array([1, 4, 6, 4, 1], dtype=np.float64) / 16.0

def weighted_moving_avg(signal, weights):
    """Apply weighted moving average. Pads edges with zeros."""
    half = len(weights) // 2
    result = np.zeros_like(signal)
    # Pad signal on both sides
    padded = np.pad(signal, half, mode='edge')
    for i in range(len(signal)):
        window = padded[i : i + len(weights)]  # extract window
        result[i] = np.dot(window, weights)     # weighted sum
    return result

smoothed_uniform    = weighted_moving_avg(signal, w_uniform)
smoothed_nonuniform = weighted_moving_avg(signal, w_nonuniform)

print('Original signal:      ', signal.astype(int))
print('Uniform avg [1/5]:    ', smoothed_uniform.astype(int))
print('Non-uniform [1,4,6,4,1]/16:', smoothed_nonuniform.astype(int))

# Also works with np.convolve (for uniform weights):
# np.convolve is cross-correlation for symmetric kernels
smoothed_np = np.convolve(signal, w_uniform, mode='same')
print('NumPy convolve result:', smoothed_np.astype(int))
```

4.5 Gaussian Filter with OpenCV

```
import cv2
import numpy as np
import matplotlib.pyplot as plt

# ──────────────────────────────────────────────────────────
# GAUSSIAN FILTER KERNEL — understanding sigma
```

```
# ──────────────────────────────────────────────────────────────

# Get the 1D Gaussian kernel that OpenCV would use (kernel size 5, sigma=1)
kernel_1d = cv2.getGaussianKernel(ksize=5, sigma=1)
# Make it 2D by outer product: kernel_2d = kernel_1d × kernel_1d^T
kernel_2d = np.outer(kernel_1d, kernel_1d)
print('5×5 Gaussian kernel (sigma=1):')
print(np.round(kernel_2d, 3))
print('Sum of all weights:', kernel_2d.sum())  # must be ≈ 1.0

# ──────────────────────────────────────────────────────────────
# APPLYING GAUSSIAN FILTER
# cv2.GaussianBlur(image, (ksize, ksize), sigmaX)
# ksize must be ODD (3, 5, 7, 9 ...)
# sigmaX=0 means OpenCV calculates sigma automatically from ksize
# ──────────────────────────────────────────────────────────────

img = cv2.imread('image.jpg', cv2.IMREAD_GRAYSCALE)
noise = np.random.normal(0, 25, img.shape).astype(np.int16)
noisy = np.clip(img.astype(np.int16) + noise, 0, 255).astype(np.uint8)

# Different sigma values — effect on blur strength
gauss_s1  = cv2.GaussianBlur(noisy, (5, 5), sigmaX=1)   # gentle
gauss_s2  = cv2.GaussianBlur(noisy, (5, 5), sigmaX=2)   # moderate
gauss_s5  = cv2.GaussianBlur(noisy, (11, 11), sigmaX=5) # heavy blur

# ──────────────────────────────────────────────────────────────
# COMPARISON: Box filter vs Gaussian filter
# ──────────────────────────────────────────────────────────────

box_5x5   = cv2.blur(noisy, (5, 5))
gauss_5x5 = cv2.GaussianBlur(noisy, (5, 5), sigmaX=1)

fig, axes = plt.subplots(2, 3, figsize=(14, 8))

axes[0,0].imshow(img,        cmap='gray'); axes[0,0].set_title('Original')
axes[0,1].imshow(noisy,      cmap='gray'); axes[0,1].set_title('Noisy (σ=25)')
```

```
axes[0,2].imshow(box_5x5,    cmap='gray'); axes[0,2].set_title('Box Filter 5×5')
axes[1,0].imshow(gauss_s1,   cmap='gray'); axes[1,0].set_title('Gaussian σ=1 (5×5)')
axes[1,1].imshow(gauss_s2,   cmap='gray'); axes[1,1].set_title('Gaussian σ=2 (5×5)')
axes[1,2].imshow(gauss_s5,   cmap='gray'); axes[1,2].set_title('Gaussian σ=5 (11×11)')

for ax in axes.flat:
    ax.axis('off')

plt.suptitle('Box Filter vs Gaussian Filter Comparison', fontsize=14, y=1.01)
plt.tight_layout()
plt.savefig('gaussian_vs_box.png', dpi=150)
plt.show()
```

4.6 Manual 2D Correlation (From Scratch)

```
import numpy as np

# ──────────────────────────────────────────────────────────
# IMPLEMENTING 2D CORRELATION FROM SCRATCH
# This is what cv2.filter2D does internally!
# h(i,j) = sum over k,l of: g(k,l) * f(i+k, j+l)
# ──────────────────────────────────────────────────────────

def correlate2d_manual(image, kernel):
    """
    Apply 2D cross-correlation manually (without OpenCV).

    Args:
        image  (np.ndarray): 2D grayscale image (H x W)
        kernel (np.ndarray): 2D kernel/filter (kH x kW)

    Returns:
        np.ndarray: output image (same size, uint8)
    """
    H, W = image.shape           # image height, width
```

```
    kH, kW = kernel.shape        # kernel height, width
    pad_h = kH // 2              # vertical padding size
    pad_w = kW // 2              # horizontal padding size

    # Pad the image with zeros on all sides
    # This ensures output has same size as input
    img_padded = np.pad(image.astype(np.float64),
                        ((pad_h, pad_h), (pad_w, pad_w)),
                        mode='constant', constant_values=0)

    output = np.zeros((H, W), dtype=np.float64)

    # Slide the kernel over every pixel
    for i in range(H):           # row index in output
        for j in range(W):       # col index in output
            # Extract the patch of same size as kernel
            patch = img_padded[i : i+kH, j : j+kW]
            # Element-wise multiply kernel & patch, then sum
            output[i, j] = np.sum(kernel * patch)

    # Clip to valid pixel range and convert
    return np.clip(output, 0, 255).astype(np.uint8)


# ── Test on a simple image ──────────────────────────
image = np.array([
    [0,  0,  0,  0,  0,  0,  0,  0,  0,  0],
    [0,  0,  0,  0,  0,  0,  0,  0,  0,  0],
    [0,  0,  0, 90, 90, 90, 90, 90,  0,  0],
    [0,  0,  0, 90, 90, 90, 90, 90,  0,  0],
    [0,  0,  0, 90, 90, 90, 90, 90,  0,  0],
    [0,  0,  0, 90,  0, 90, 90, 90,  0,  0],
    [0,  0,  0, 90, 90, 90, 90, 90,  0,  0],
    [0,  0,  0,  0,  0,  0,  0,  0,  0,  0],
    [0,  0, 90,  0,  0,  0,  0,  0,  0,  0],  # <-- outlier!
    [0,  0,  0,  0,  0,  0,  0,  0,  0,  0],
], dtype=np.uint8)
```

```
# 3x3 box filter
box_kernel = np.ones((3, 3), dtype=np.float64) / 9.0

# Apply our manual correlation
result = correlate2d_manual(image, box_kernel)

print('Input image:')
print(image)
print('After 3x3 box filter:')
print(result)

# Verify against OpenCV
import cv2
result_cv = cv2.blur(image, (3, 3))
print('OpenCV result matches:', np.allclose(result, result_cv))
```

4.7 Summary: All Filters in One Script

```
import cv2
import numpy as np
import matplotlib.pyplot as plt

# Load image and add noise
img   = cv2.imread('image.jpg', cv2.IMREAD_GRAYSCALE)
noise = np.random.normal(0, 20, img.shape).astype(np.int16)
noisy = np.clip(img.astype(np.int16) + noise, 0, 255).astype(np.uint8)

# ── ALL FILTER OPTIONS ──────────────────────────────

box_3   = cv2.blur(noisy, (3, 3))           # Box 3×3
box_5   = cv2.blur(noisy, (5, 5))           # Box 5×5
gauss_3 = cv2.GaussianBlur(noisy, (3,3), 0) # Gaussian 3×3
gauss_5 = cv2.GaussianBlur(noisy, (5,5), 1) # Gaussian 5×5 σ=1
gauss_9 = cv2.GaussianBlur(noisy,(11,11), 3) # Gaussian big σ=3
median  = cv2.medianBlur(noisy, 5)           # Median (bonus!)
```

```
# ── DISPLAY ALL ──────────────────────────────────────────
fig, axes = plt.subplots(2, 4, figsize=(18, 9))
results = [img, noisy, box_3, box_5,
           gauss_3, gauss_5, gauss_9, median]
labels  = ['Original', 'Noisy', 'Box 3×3', 'Box 5×5',
           'Gauss 3×3', 'Gauss 5×5 σ=1', 'Gauss 11×11 σ=3', 'Median 5×5']

for ax, res, lbl in zip(axes.flat, results, labels):
    ax.imshow(res, cmap='gray', vmin=0, vmax=255)
    ax.set_title(lbl, fontsize=10)
    ax.axis('off')

plt.suptitle('Complete Filter Comparison', fontsize=14)
plt.tight_layout()
plt.savefig('all_filters.png', dpi=150, bbox_inches='tight')
plt.show()
```

5. Complete Summary

Key Concepts at a Glance

Image Noise:

- Formula: $\hat{I}(x,y) = I(x,y) + n(x,y)$
- Noise is random, independent per pixel, zero-mean.
- True pixel values are locally smooth (similar to neighbours).

Noise Removal Strategy:

- Average a pixel with its neighbourhood → true signal reinforces, noise cancels.
- Larger window = more noise removal, but more blur.
- Weighted average: closer pixels contribute more.

Filters:

- Box filter: uniform weights → fast, blocky blur.
- Gaussian filter: bell-curve weights → smooth natural blur, no ringing.
- Both are examples of cross-correlation: $h = g \otimes f$

Filter / Concept	OpenCV Function
Box (averaging) filter	cv2.blur(img, (kW, kH))
Gaussian filter	cv2.GaussianBlur(img, (kW,kH), sigma)
Custom kernel correlation	cv2.filter2D(img, -1, kernel)
Get Gaussian kernel	cv2.getGaussianKernel(ksize, sigma)
Median filter (bonus)	cv2.medianBlur(img, ksize)
Add Gaussian noise (NumPy)	np.random.normal(0, sigma, shape)

Next Lecture

Edge Detection — how to find the boundaries between regions in an image. We will build on filtering concepts: a derivative filter highlights edges instead of smoothing them.